

Task 3 — Model Deployment

Heart Disease Prediction API using Flask and Docker
InternSpark Artificial Intelligence Internship

1. Project Overview

This project deploys a trained Random Forest machine learning model as a Flask REST API and packages the application using Docker. The API accepts 13 patient features and returns a binary heart disease prediction.

2. Objective

The objective is to expose the trained machine learning model through a REST API and containerize the application so it can run in a consistent Docker environment.

3. Technologies Used

Python 3.14, Flask 3.1.3, NumPy 2.5.3, Scikit-learn 1.9.0, Docker, Docker Desktop, and WSL 2.

4. Project Files

app.py — Flask API application.
heart_disease_model.pkl — trained Random Forest model.
scaler.pkl — feature scaler used during model training.
requirements.txt — Python dependencies.
Dockerfile — Docker image configuration.
README.md — project documentation and API usage.

5. API Endpoints

GET / confirms that the API is running.

POST /predict accepts 13 input features in JSON format and returns the prediction. The saved feature scaler is applied before model prediction.

6. Example Prediction

Example input: {"features": [63, 1, 3, 145, 233, 1, 0, 150, 0, 2, 3, 0, 0, 1]}

Example output: {"prediction": 0}

7. Docker Implementation

The Dockerfile uses Python 3.14 slim, installs pinned dependencies, copies the Flask application and saved model/scaler files, exposes port 5000, and starts the application with Python.

8. Container Verification

The Docker image was successfully built as heart-disease-api. The container heart-disease-container was started with port mapping 0.0.0.0:5000 → 5000/tcp and verified using docker ps.

9. API Test

The deployed container was tested through POST /predict using PowerShell. The request returned prediction 0, confirming that the API responded successfully.

10. Conclusion

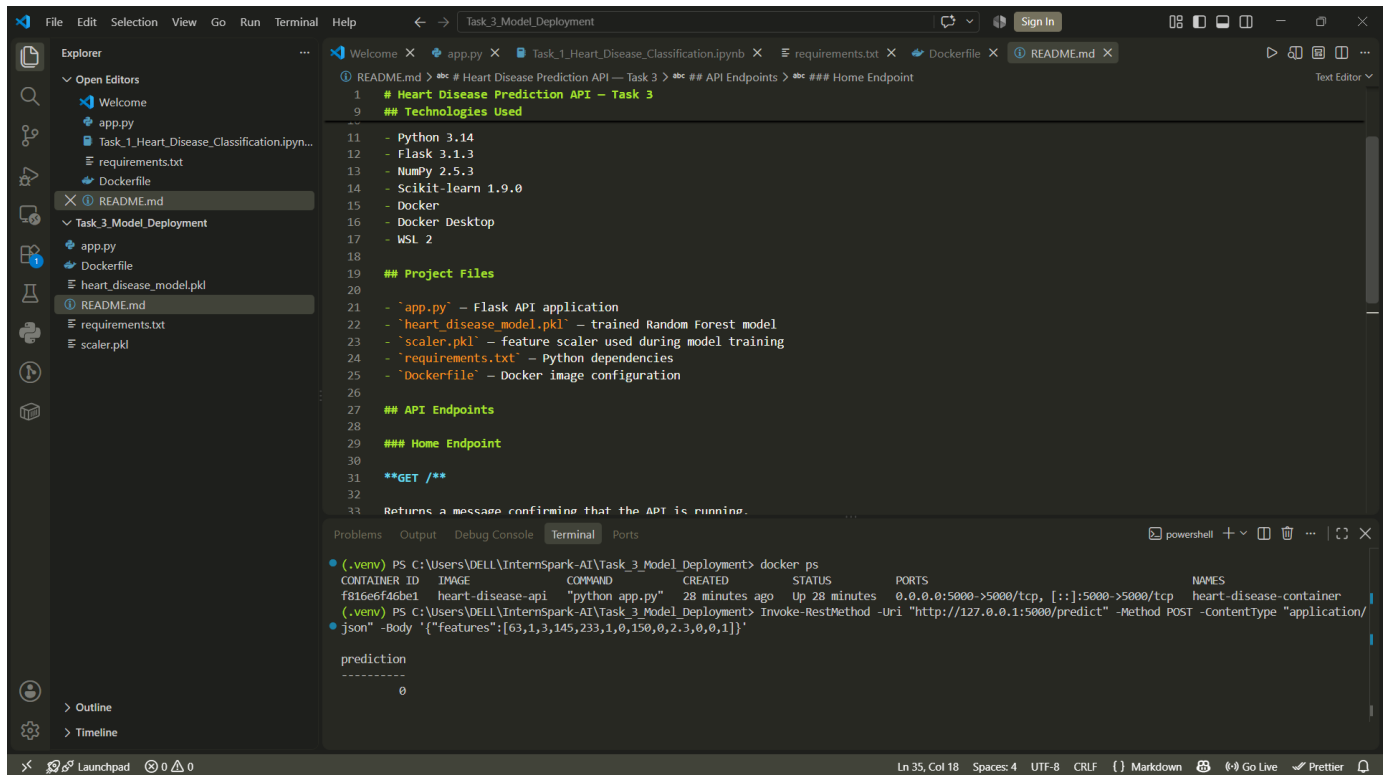
Task 3 demonstrates deployment of the trained Random Forest model through a Flask REST API and Docker containerization. The application files, model, scaler, dependencies, Dockerfile, and documentation are included in the repository.

11. GitHub Repository

<https://github.com/shani-7718/InternSpark-AI-Task-3>

12. Deployment Evidence

The screenshot below shows the running Docker container and the successful prediction request.



The screenshot displays a Visual Studio Code editor with a project named "Task_3_Model_Deployment". The Explorer sidebar on the left shows the project structure, including files like "app.py", "requirements.txt", "Dockerfile", "heart_disease_model.pkl", and "README.md". The main editor window shows the "README.md" file, which contains the following content:

```
1 # Heart Disease Prediction API - Task 3
2
3 ## Technologies Used
4
5 - Python 3.14
6 - Flask 3.1.3
7 - NumPy 2.5.3
8 - Scikit-learn 1.9.0
9 - Docker
10 - Docker Desktop
11 - WSL 2
12
13 ## Project Files
14
15 - 'app.py' - Flask API application
16 - 'heart_disease_model.pkl' - trained Random Forest model
17 - 'scaler.pkl' - feature scaler used during model training
18 - 'requirements.txt' - Python dependencies
19 - 'Dockerfile' - Docker image configuration
20
21 ## API Endpoints
22
23 ### Home Endpoint
24
25 **GET /**
26
27 Returns a message confirming that the API is running.
```

The terminal window at the bottom shows the command to run the Docker container and the successful prediction request:

```
(.venv) PS C:\Users\DELL\InternSpark-AI\Task_3_Model_Deployment> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
f816e6f46be1  heart-disease-api                   "python app.py"         28 minutes ago Up 28 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  heart-disease-container

(.venv) PS C:\Users\DELL\InternSpark-AI\Task_3_Model_Deployment> Invoke-RestMethod -Uri "http://127.0.0.1:5000/predict" -Method POST -ContentType "application/json" -Body '{"features": [63, 1, 3, 145, 233, 1, 0, 150, 0, 2, 3, 0, 0, 1]}'

prediction
-----
0
```